Compile the Conversation of Multi-Agent Coordination: Provably Safe and More Token-Efficient

Anonymous authors
Paper under double-blind review

Abstract

Skill files, agent markdowns, and specification documents have become the de facto programming language of multi-agent systems—and this language ships without a compiler. Whether a set of skills can deadlock, leak a confidential payload, or ever reach its goal is today discoverable only by running the system or judging its traces, and neither can establish the *absence* of failure: a thousand clean runs warrant nothing about run 1,001. We present STJP, a static compiler for agent conversations grounded in multiparty session types (MPST): an LLM drafts the user's intent as a Scribble global protocol; the protocol is validated *before any agent runs* (deadlock-freedom, communication safety, session fidelity, goal reachability by construction); the validated type is then projected into per-role lean skills and mechanically compiled into deterministic monitors, value-dependent choice guards, and an EFSM scheduler—code, not agents. Unlike concurrent work that obtains guarantees by generating the coordination layer as trusted code around opaque LLM calls, STJP enforces the protocol on *free-running, untrusted* LLM agents at the message boundary. Across pre-registered experiments (two model families, up to seven ablation arms, a 1,200-trial validation suite, and nine cases built from real, unmodified public skill files) the compiled arm is simultaneously the safest configuration and the most token-efficient one, because every token class the unenforced arms waste is a class the compiler removes. **(i)** *Projection replaces prose.* A textual plan is re-read by every role in every round; a projected skill carries only that role's slice—the same work at 63% fewer tokens, with the structural gap widening from $9.2\times$ to $17.1\times$ between two and ten roles. **(ii)** *The scheduler replaces polling.* Driving the runtime from the validated type cuts LLM calls by 73% ($9\times$ lower cost-to-goal), and once violated runs stop counting as successes the compiled arm wins by 4–$22\times$ on cost-to-*clean*-goal—while committing zero unauthorized irreversible acts at every model tier, where unenforced arms commit up to 95 in 100 trials. **(iii)** *Enforcement deletes the most expensive token class: failure.* Unmodified real skill files deadlock, stall, or succeed only as a model-dependent coin flip in all nine live cases; on a looping code-review protocol the plan-as-text arm *livelocks*, burning its entire 16-round budget in 10/10 trials at $3.6\times$ the tokens of the compiled arm that ships the merge; and a seeded circular-wait protocol is rejected statically at *zero* token cost—the cheapest failure is the one that never runs. The instruments are themselves tested: the checker detects 95.1% of seeded well-formedness defects with *zero* false positives, and the structural gate plus value refinements block 100% of 1,200 injected exfiltration attempts (keyword rules: 41.7%). Three typed extensions—stateful session invariants, a decidable subtype build check, and statically verified crash handling—extend coverage; conformance and goal achievement are defined mechanically over a typed event log, needing neither golden trajectories nor LLM judges; falsified predictions are reported prominently; the suite is released as a benchmark. Prompted compliance is a behavior; compiled enforcement is a property—and the property spends fewer tokens.

1 Introduction

Practitioners now steer LLM agents with markdown artifacts: Anthropic's Agent Skills open standard (`SKILL.md`), `AGENTS.md` conventions, OpenAI's Model Spec, MCP tool manifests, and enterprise "agent spec" documents [34]. Functionally, these artifacts are programs. They fix an interaction discipline—which role sends which message to which role, in what order, carrying what payload, with which authorization preceding which irreversible act. When several agents must cooperate (a planner, an analyst, an auditor, a filer), the artifact is not a prompt; it is a *protocol*.

Yet the toolchain that grew around ordinary programming languages is absent. There is no parser that rejects an incoherent skill, no type checker that proves two skills cannot mutually block, no compiler that guarantees the declared goal is reachable. Format linting exists—Skilldex reports that 34% of community skill packages violate even the published format specification [27]—but format validity says nothing about interaction soundness. A `SKILL.md` can parse perfectly while describing a choreography in which the planner waits for a result the worker will never send,

because the worker is waiting for an answer the planner cannot give. Nothing crashes; the system sits there, emitting tokens.

The field's current answers are all *dynamic*. Runtime guardrails such as AgentSpec [23] and Agent-C [24] intercept unsafe tool calls with rule DSLs or SMT-checked temporal constraints; benchmarks such as ST-WebAgentBench [25] and BeSafe-Bench [26] measure, after the fact, how often agents violated policy; LLM-as-judge pipelines score traces with a second model whose own reliability is contested (reported true-negative rates below 25% and expert agreement of 64–68% [33]). All of these *observe executions*. None can certify a *specification*. Testing, as Dijkstra observed, shows the presence of bugs, never their absence.

This paper poses the question in its static form: **can we type-check the conversation itself, before any agent runs?** Our answer is STJP (Session-Typed skills/Judge Protocol), a compiler pipeline built on two decades of multiparty session type theory [1, 2] and its industrial protocol language, Scribble [Scribble]. The user states an intent; an LLM drafts a Scribble global protocol; the Scribble checker validates well-formedness statically—rejecting deadlock precursors, incoherent choice, unguarded recursion, unreachable goals—and only a validated protocol is projected, role by role, into local contracts that become each agent's lean skill. From the same projection we mechanically generate a deterministic finite-state monitor per role, value-dependent *choice guards* in the asserted-MPST style [4], and an EFSM scheduler. Enforcement and evaluation are plain code: bit-reproducible, token-free, judge-free. Concurrent work obtains coordination guarantees by taking communication away from the agents—the coordination layer is generated as trusted code around opaque LLM calls [17]; STJP leaves the agents free and makes the boundary trustworthy.

Contributions.

1. **A reframing, made precise.** Skills, specs, and agent markdowns constitute an untyped protocol language; we give, to our knowledge, the first end-to-end pipeline that statically type-checks these *deployed artifacts* via MPST and enforces the result on *free-running, untrusted* LLM agents before and during execution—in contrast to concurrent work that obtains guarantees by generating the coordination layer as trusted code around opaque LLM calls [17] (§3–§4).
2. **Judge-free evaluation and the CGC metric.** Because every run is, by construction, a path through a validated global type, *conformance* and *goal achievement* receive mechanical definitions over a typed event log—no golden trajectory, no LLM judge—together with *clean-goal completion* (goal reached with zero violations) and a consequence-graded severity taxonomy (S0–S4) validated empirically: $P(\text{goal failure} \mid \text{S2+ violation}) = 100\%$ in every arm of every run (§5).
3. **Three empirical regularities** that goal-rate metrics hide (§6–§7, Figs. 2–4): prose compliance does not survive concurrency (100% goal rate with 95/100 unauthorized irreversible acts); unenforced safety is capability-dependent while enforcement is capability-invariant (95→0→0 premature filings vs. zero at every tier); and the enforcing stack is simultaneously the safest and the cheapest arm ($9\times$ cost-to-goal, 4–$22\times$ cost-to-clean-goal) once the projection drives the runtime. A statically rejected deadlock exhibits the failure class no runtime method can rule out. We also report the run in which unenforced protocol text *ties* the enforcement gate, and what that bounds. Nine further cases assembled from unmodified public skill files replicate the regularities in the wild and add a fourth: on looping protocols, a textual plan can fail *both* ways—deadlock without it, stable livelock with it—while the compiled arm ships at $3.6\times$ less than the cost of the failure (§7).
4. **Guarantee transfer.** We restate the MPST metatheory—communication safety, session fidelity, deadlock-freedom, monitorability, refinement, gradual typing of the LLM endpoint—as agent-system properties with direct safety and cost consequences (§3, Table 2).
5. **A benchmark, released and self-tested.** Protocol-mutation testing, adversarial injection, model-strength sweeps, pass^k reliability, and translation-fidelity measurement, executed at $n{=}100$ with pre-registered predictions graded—including the falsified ones (§7). We additionally release the instruments that turn the intent-to-protocol translation step (which we call, for brevity, the *seam*) into a verifier-in-the-loop learning problem—a validated Scribble grammar, an EFSM-equivalence scorer, a memoryless faithfulness panel, an EFSM-deduplicated protocol corpus, and a preregistered training protocol (H1–H6)—each measured before any training; the training runs themselves are the declared next step, and this paper ships a results template into which their numbers drop without restructuring (§8).

Table 1: Positioning. STJP is, to our knowledge, the only system that both establishes guarantees *before* execution and enforces them on *untrusted* agents at runtime, with evaluation requiring neither a reference trajectory nor an LLM judge. "partial" = expressible per hand-written rule, without a global theorem. "n/a†": in Bollig et al. [17] coordination is trusted generated code and agents cannot address messages at all, so recipient legality is moot rather than checked; the guarantee does not transfer to runtimes where agents send messages themselves.

	When it acts	Deadlock-freedom	Goal reachability	Unauth. recip. blocked	Value/branch guards	Judge-free metrics
LLM-as-judge	post hoc	no	no	no	no	no
Rule guards [23]	runtime	no	no	partial	partial	no
Temporal/SMT [24]	runtime	no	no	partial	partial	no
Graph frameworks (LangGraph)	wiring	no	no	no	no	no
MSC codegen [17]	compile (codegen)	static	no	n/a†	no	no
STJP (this work)	**compile + runtime**	**static**	**static**	**typed**	**yes**	**yes**

2 Related Work

Agent governance layers. Recent syntheses organize agent governance as a three-layer stack [28, 29]: Layer 0 checks that a specification parses; Layer 1 asks whether the specification is semantically sound ("if the agent believes the spec, will it be correct?"); Layer 2 measures whether runtime behavior matches claims. Layer 0 is commoditized [27]. Layer 2 is an active benchmark frontier: the best of 13 agents evaluated by BeSafe-Bench completes fewer than 40% of tasks while fully respecting safety constraints [26]; ST-WebAgentBench's Completion-under-Policy shows policy-respecting completion at under two-thirds of nominal completion [25]. Layer 1 is, in those surveys' words, "not available." STJP occupies it. Regulatory pressure points the same way: path-based analyses of runtime governance note that the EU AI Act's high-risk provisions, effective August 2026, demand demonstrably governed orchestration [22].

Runtime enforcement. AgentSpec [23] enforces trigger/predicate/action rules around tool calls and prevents >90% of unsafe executions in its evaluation; Agent-C [24] compiles temporal constraints to SMT checks interleaved with generation; LGA [29] composes sandboxing, intent verification, zero-trust authorization, and audit logging. All act per execution. Rule DSLs cannot express, let alone verify, *global* interaction properties—deadlock-freedom, goal reachability, choice coherence—because these are properties of the protocol structure, not of any single call. STJP rejects the faulty specification before the first LLM call, the only point at which a guarantee can be universal rather than per-run; its runtime gate then enforces exactly the properties the static phase proved enforceable [10]. We are explicit about where the novelty lies: the session-type theory we build on is three decades deep and is assembled here, not invented—our contributions are the compiler framing (skills *are* an untyped protocol language; STJP is its missing compiler), the gradual-typing integration of an untyped LLM participant, the observation that the *scheduler* is itself a projection artifact, and the judge-free empirical program the rest of this paper executes.

Session types and choreographies. MPST [1, 2] and its toolchain Scribble [Scribble] give protocols a type theory (§3); choreographic programming [15, 16] generates endpoint code from a global script by projection. An `AGENTS.md` is an informal choreography; STJP makes it formal and lets projection do the work. Prior NL→formal-spec pipelines (NL2Spec [30], PROSPER [31]) target LTL and network RFCs; we target a protocol algebra whose checker exists and whose theorems transfer.

Projection-based coordination for LLM agents. Concurrently with this work, Bollig et al. [17] introduce an MSC-based DSL (ZipperGen) whose syntax-directed projection yields deadlock-free local programs by construction, with the metatheory machine-checked in Lean 4—a mechanization discipline we do not match and genuinely admire; the classical MPST theorems STJP inherits are instead backed by three decades of peer-reviewed proof, including the monitoring theorem for *untrusted* endpoints [10] that the closed setting of Bollig et al. [17] does not require. The two systems are separated by the trust boundary. In ZipperGen the LLM never holds the send keys: coordination is executed by trusted generated code (framework-owned threads and FIFO queues), and LLM calls are opaque typed functions inside it; deadlock-freedom holds because the untrusted component is structurally incapable of sending a message at all. That is a clean closed-world result—and it is precisely why it cannot govern the deployed agent ecosystem, where free-running LLM agents (skill-driven assistants, MCP tool users, group chats, graph-framework agents) *are* the message-senders. STJP targets that open regime: the LLM is the untyped participant in the sense of

gradual session types [11], the generated monitor is the cast at the boundary, and the operative theorem is monitorability [10]. Every phenomenon our empirical program measures—95/100 premature filings under concurrent scheduling, 22 intent-only irreversible acts under the stronger model, 1,200 injected exfiltration attempts—exists only in the open regime; Bollig et al. [17] report no empirical evaluation. The systems also differ on *when* assurance is paid for: ZipperGen's runtime planner lets an LLM generate sub-workflows that inherit the structural guarantees, but structure is all its DSL encodes—goal reachability, payload refinements, authorization ordering, and recipient confidentiality are absent (data-level verification is explicitly deferred)—so a generated workflow that is structurally sound yet wrong in intent is discovered mid-execution, tokens already spent, with no human endorsement point. STJP front-loads every checkable property to compile time, where a rejected draft costs zero tokens and the human endorses G before any agent runs (§4.1); guarantees are purchased before spend, not during it. The projected artifacts differ accordingly: ZipperGen emits local Python bound to its runtime, whereas STJP emits the ecosystem's lingua franca—lean markdown contracts consumable by any agent runtime—alongside monitors that the E7 portability harness shows agreeing across independent runtimes (§7). We also note the automata-theoretic line of Li et al. [18, 19], Li & Wies [20] on complete and certified MPST projection, a natural upgrade path for stages S2–S3, and the trace-based assurance framework of Paduraru et al. [21], which, like all runtime observation, is downstream of the static question posed here.

Trace evaluation. Tool-and-step metrics (LangSmith, DeepEval, Phoenix, MLflow), process-utility metrics (TRACE [32]), policy-conditioned metrics (CuP [25]), and reliability metrics (pass^k) all presuppose a hand-authored reference trajectory or an LLM judge, because an unconstrained agent leaves no structure to score a run against. Under STJP the validated global type *is* the reference (§5).

3 Multiparty Session Types: Syntax, Semantics, Guarantees

We summarize the fragment of MPST that STJP compiles to, in the modern formulation [1, 2]; this section fixes notation and states (without proof) the theorems whose transfer to agents Table 2 prices.

3.1 Global and local types

Fix a set of *roles* $p, q, r \in \mathcal{R}$ (agents, tools, the orchestrator), *labels* $\ell \in \mathcal{L}$ (message kinds), and *payload sorts* U (base sorts and refinements, below). *Global types* describe the whole conversation from a bird's-eye view; *local types* describe one role's view of it:

$$\mathsf{G} \;::=\; p \to q : \{\ell_i(U_i).\,\mathsf{G}_i\}_{i\in I} \;\mid\; \mu t.\,\mathsf{G} \;\mid\; t \;\mid\; \mathtt{end} \qquad \text{(global types)}$$

$$\mathsf{T} \;::=\; q \oplus \{\ell_i(U_i).\,\mathsf{T}_i\}_{i\in I} \;\mid\; p \,\&\, \{\ell_i(U_i).\,\mathsf{T}_i\}_{i\in I} \;\mid\; \mu t.\,\mathsf{T} \;\mid\; t \;\mid\; \mathtt{end} \qquad \text{(local types)}$$

$p \to q : \{\ell_i(U_i).\mathsf{G}_i\}_i$ reads: p chooses one label ℓ_i, sends it with a payload of sort U_i to q, and the conversation continues as G_i. Dually, $q \oplus \{\ldots\}$ is an *internal choice* (this role selects and sends to q) and $p \,\&\, \{\ldots\}$ an *external choice* (this role branches on what it receives from p). $\mu t.\mathsf{G}$ is guarded recursion (loops); $\mathtt{end}$ terminates the session. In the agent reading: labels are message kinds (`ExpenseData`, `Approval`), sorts are payload schemas, branching is a decision (audit vs. standard path), recursion is a bounded retry/deliberation loop.

3.2 Projection

The *projection* $\mathsf{G} \upharpoonright r$ extracts role r's local type from G:

$$\big(p \to q : \{\ell_i(U_i).\mathsf{G}_i\}_{i\in I}\big) \upharpoonright r \;=\; \begin{cases} q \oplus \{\ell_i(U_i).\,\mathsf{G}_i \upharpoonright r\}_{i\in I} & \text{if } r = p \\ p \,\&\, \{\ell_i(U_i).\,\mathsf{G}_i \upharpoonright r\}_{i\in I} & \text{if } r = q \\ \sqcap_{i\in I}\, \mathsf{G}_i \upharpoonright r & \text{otherwise,} \end{cases}$$

with $(\mu t.\mathsf{G}) \upharpoonright r = \mu t.(\mathsf{G} \upharpoonright r)$ when r occurs in G (else $\mathtt{end}$), $t \upharpoonright r = t$, $\mathtt{end} \upharpoonright r = \mathtt{end}$. The *merge* $\sqcap$ requires that a role not involved in a choice either behaves identically on all branches or is informed of the outcome before its behavior may differ; if the merge is undefined, G is *rejected as unprojectable*—already a useful lint: it flags protocols in which an agent's duties silently depend on a decision it was never told about. Concurrent MSC-based work removes

this side condition by construction, inserting owner-side control broadcasts so that projection is total [17]; we keep the merge check deliberately: in the skills setting an undefined merge is diagnostic signal—it names the role whose obligations depend on a choice it cannot observe—and the repair (informing that role) changes who learns what, a design decision the human should endorse at S1, not one the compiler makes silently. A global type is *well-formed* if it is projectable onto every role and all recursion is guarded. Each local type induces a finite *endpoint state machine* (EFSM) $M_r = (S, s_0, \Sigma, \delta, F)$ whose alphabet is r's sends and receives; conversely, under well-formedness, the (associative) composition of the projected EFSMs recovers the behaviors of G [13].

3.3 Operational semantics and typing

A *configuration* C pairs role processes with FIFO queues (asynchrony); actions $pq!\ell(v)$ / $pq?\ell(v)$ enqueue and dequeue, and $\mathrm{traces}(C)$ collects finite action sequences. The typing judgment $\Gamma \vdash C \triangleright \Delta$ assigns each role its local type; in the "Less is More" formulation [2], behavioral properties are checked directly on the typing environment, repairing soundness gaps of the classical presentation and enlarging the set of typable protocols. Well-typedness is preserved under reduction (subject reduction), which drives:

- **T1 (Communication safety.)** In a well-typed configuration, no role ever receives a message whose label or payload sort it cannot handle at its current state: every send is matched by a compatible receive [1].
- **T2 (Session fidelity / protocol conformance.)** Every trace of a well-typed configuration is a trace of G: execution *refines* the global type [1, 2].
- **T3 (Deadlock-freedom / progress.)** A well-typed single-session configuration never reaches a stuck non-final state: some role can always move until `end` [1, 2].
- **T4 (Monitorability.)** If every endpoint is wrapped by a monitor that enforces its projected local type $\mathsf{G} \upharpoonright r$—accepting, suppressing, or rejecting boundary messages—then the observable behavior of the whole (untrusted) network satisfies G; local checks compose to a global guarantee with no central observer [10].

T4 is the license for STJP's architecture: LLM endpoints are untrusted, monitors are generated from projections, and *global* compliance falls out of *local*, $O(1)$-per-event checking.

3.4 Extensions STJP uses

Refinements and choice guards. Asserted/refined MPST [4, 9] decorates payloads with predicates, $\ell(x{:}U)\{A\}$, where A may mention previously bound values; Chen & Honda [5] extend this line to *stateful* asynchronous properties—assertions over virtual state that evolves across the whole conversation, checkable at endpoints under asynchrony—implemented and measured as STJP's *session ledger* (§9); and decorates *choices* with guards selecting which branch the data permits: $p \to q : \{\, \ell_i(x_i{:}U_i)\{A_i\}.\ \mathsf{G}_i \,\}_i$ with A_i over the session history. Statically dischargeable guards go to an SMT check; guards over LLM-produced values compile into the monitor. This is precisely the construct that closes the "protocol-legal but value-wrong" hole we observed empirically (§6): choosing the `Standard` branch on $50k+ revenue is legal for classical MPST and a violation for guarded MPST.

Behavioral flexibility via precise subtyping. An agent need not follow its projected local type *exactly*: session subtyping characterizes every safe deviation, and its *precise* (sound & complete) characterization [6, 7] draws the line tightly—every subtype-conformant behavior is safe, and any behavior outside the relation fails in some context. This is the formal license behind our lean contracts (§7): a compressed contract refined by the verbose one loses nothing, and preciseness says exactly how much slack a monitor may lawfully grant before it must reject; both directions—a compile-time subtype check and a tolerance-bounded gate—are implemented and measured in §9. **Gradual typing of the LLM endpoint.** Gradual session types [11] mix statically typed participants with a dynamically typed (`dyn`) one, inserting casts at the boundary that preserve safety and progress. The LLM is STJP's `dyn` participant; the generated monitor *is* the cast. The theory predicts what we measure: the untyped participant may *attempt* anything; nothing ill-typed crosses the boundary.

Beyond the fragment. Parameterised MPST [14] types fan-out over n workers; dynamic multirole session types [12] admit roles joining/leaving—both on our roadmap, neither exercised by the experiments below, which fix the role set.

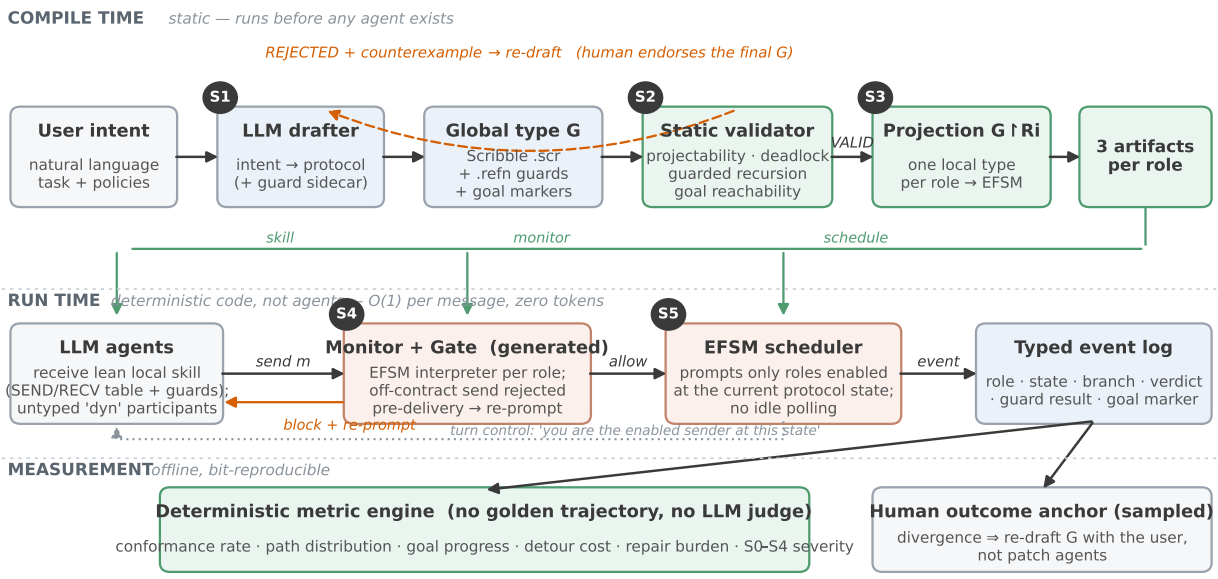

Figure 1: The STJP pipeline. Compile time (top): an LLM drafts the user's intent as a Scribble global type with a refinement/choice-guard sidecar; the static validator rejects unsafe drafts with counterexamples (drafting the banking protocol live, four consecutive LLM drafts were rejected before the fifth passed); only a validated G is projected. Projection emits three artifacts per role: a lean local contract (the agent's skill), a generated EFSM monitor, and the schedule. Run time (bottom): the gate rejects off-contract sends *pre-delivery* and re-prompts; the scheduler prompts only roles enabled at the current protocol state; every message lands in a typed event log over which all metrics are deterministic database queries. One sampled human outcome anchor stays outside the metric family.

4 The STJP Compiler

4.1 Pipeline

S1 — Intent → draft protocol. The user states a task ("six agents process a revenue report; if revenue exceeds $50,000 an audit is mandatory before filing"). An LLM drafts a Scribble global protocol—roles, labels, payload sorts, branches, guarded recursion—plus a sidecar of refinement and choice guards (`.refn`; stock `scribble-java` is untouched) and *goal markers*: designated protocol points whose ordered satisfaction encodes task completion, one marked FINAL. The draft is endorsed by the user; intent negotiation is human-in-the-loop by design.

S2 — Static validation. The Scribble checker validates well-formedness: projectability (choice coherence via the merge $\sqcap$), matched sends/receives, guarded recursion, and reachability of every goal marker (the achievability check emits a witness path, later reused as the efficiency baseline). Validation is not decorative: live-drafting the banking case, four consecutive LLM drafts were rejected before the fifth passed, and the rejected finance draft contained a wait-for cycle $[\mathit{Writer} \to \mathit{RevenueAnalyst} \to \mathit{TaxVerifier}]$ that *would* deadlock at runtime—caught before any agent ran. Agents run on that unsafe draft completed only 20% of trials.

S3 — Projection → local contracts. $\mathsf{G} \upharpoonright r$ is rendered as a lean per-agent skill: a SEND/RECV table over the role's EFSM states with its refinement and choice guards attached at the states where the decision is made. The compressed contract is often *smaller than the prose it replaces* (banking: 1,063–1,986 characters vs. 1,870 for the intent-only prompt): the table substitutes for coordination guesswork.

S4 — Monitor generation. Each local type compiles mechanically to its EFSM; the runtime monitor for role r is an interpreter of M_r with a state pointer and a value store for guard evaluation—a small generated Python program, deterministic, $O(1)$ per event, no LLM call. Verdicts: *allow* / *repair* / *block*, plus `choice_guard_violation`. In *gate* mode, off-contract sends are rejected *pre-delivery* and the role is re-prompted with its enabled actions; by T4 these purely local gates jointly enforce G.

S5 — EFSM scheduling. The same automaton knows whose turn it is. Instead of polling every agent every round (each idle poll a full LLM call answering WAIT), the scheduler prompts only roles enabled at the current protocol state,

Table 2: Static guarantees inherited from MPST (§3) and their operational meaning.

Guarantee	Agent-system meaning	Consequence
Deadlock-freedom (T3)	Circular waits rejected at compile time	Kills the silent-stall, token-burning failure mode for *all* runs
Communication safety (T1)	An agent can only emit messages the protocol permits at its state; recipient sets are part of the type	Off-protocol detours and unauthorized disclosure rejected by construction
Session fidelity (T2)	Every trace refines the endorsed G	The log is audit evidence against a proven reference
Monitorability (T4)	Local $O(1)$ gates compose to global compliance	Decentralized enforcement; no watching agents
Bounded recursion	Loops carry guarded bounds	No infinite-loop billing; the bound is auditable pre-deploy
Bounded messages	Worst-case per-role message count from projection	Token cost becomes a ceiling, not a hope
Minimal capabilities	Required tools = labels of $\mathsf{G}\upharpoonright r$	Least-privilege `allowed-tools` derived, not guessed
Refinements + choice guards	Payload and branch predicates checked at the boundary	"Protocol-legal but value-wrong" branches (skip audit on $50k+) become violations

and tells sender-only roles so at the point of decision. This targets the dominant cost (the turn loop: $\sim$4.5 calls per delivered message unscheduled) and the dominant non-safety failure (liveness stalls, §6).

4.2 The typed event log and judge-free metrics

The monitor emits, per message, a typed event: role, protocol position, branch choice, gate verdict, guard outcomes, goal-marker state. Over this log, evaluation is counting, not judging: conformance rate (allowed/repaired/blocked, per message—Completion-under-Policy computed mechanically); path distribution (each run reduces to a named path through G; a retry-loop entry rate jumping 9%$\rightarrow$40% after a change is a regression no golden trajectory could encode); goal progress (read off $\checkmark\varphi$ markers—the deterministic replacement for LLM-judged "milestone completion," and the only metric class that sees an agent looping in place); detour cost (events $\div$ the compile-time witness's shortest accepting path); repair burden (gate interventions over time). All are bit-reproducible: re-running the evaluators on a run directory yields identical numbers although the measured agents are stochastic.

5 Evaluation Methodology

5.1 Defining conformance and goal achievement

Conformance, three levels. Let T be a run's trace. *(C-struct)* Structural: every message of T is accepted by the corresponding monitor EFSM—equivalently T is a path through the validated G (deterministic, per-message). *(C-refine)* Refinement: all payload and choice-guard predicates hold. *(C-sem)* Semantic: payload content is faithful to the task; only this residue may require an LLM judge, run batched, sampled, calibrated on human labels, and stored apart from the deterministic verdicts. Design principle: shrink the judged surface to what is genuinely semantic.

Goal achievement. A run achieves its goals *strictly* iff every goal marker is satisfied, in protocol order and in the *same attempt*, including the FINAL marker, under three critical-property checks: **C1 data provenance** (reported values trace to real inputs), **C2 context completeness** (required reads precede the decision, by timestamp), **C3 authorization before irreversible acts** (`approve` precedes `file`, by message order). Relaxed rungs (role-pair, semantic) are reported alongside. We additionally report **clean-goal completion (CGC)**: the fraction of trials that reach the goal with *zero* violations of any severity. GCR asks "did it get there"; CGC asks "did it get there without breaking anything on the way"—the gap between them is precisely the lucky-run phenomenon that pass-rate metrics hide. One sampled human-labeled outcome measure stays *outside* the family as an anchor: a perfectly conformant system can execute

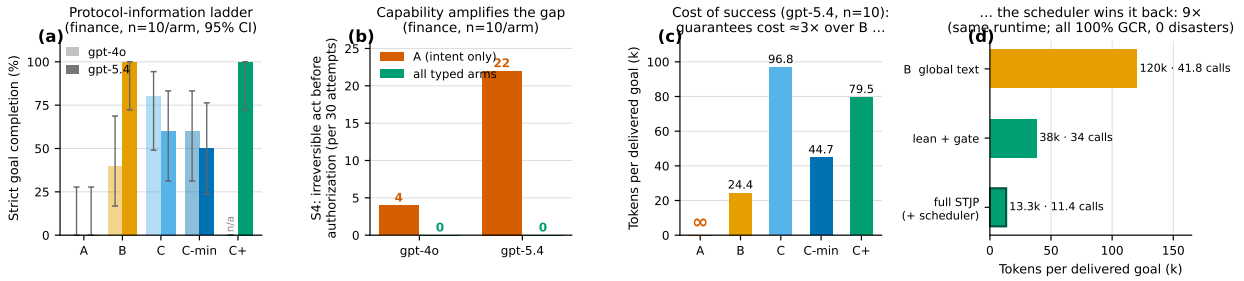

Figure 2: Headline results. (a) Strict goal completion along the protocol-information ladder for both models. (b) Disasters: intent-only agents under the *stronger* model committed 22 unauthorized irreversible acts in 30 attempts; every typed arm, zero. (c) Cost of one delivered goal (gpt-5.4 grand run): unenforced protocol text (B) is cheapest when the model happens to comply—guarantees cost $\approx 3\times$ over B on this run. (d) The EFSM scheduler recovers the cost: on a shared decentralized runtime, full STJP delivers the same 100%/zero-disaster outcome at 13.3k tokens per goal vs. 120k for protocol-as-text ($9\times$) with -73% LLM calls; tokens-per-call stays flat, so the entire saving is fewer calls.

Table 3: Protocol-information ladder, finance case, gpt-4o, n=10 per arm.

Metric	A intent	B global text	C local	C-min
GCR (strict)	0%	40%	**80%**	60%
Monitor acceptance	0%	60%	**100%**	84%
Tokens/trial	15.4k	35.4k	33.7k	28.2k
Cost of success (tokens ÷ GCR)	∞	88.5k	**42.1k**	47.0k
Redundancy ratio (events ÷ min. path)	1.9×	2.2×	**1.5**×	2.0×
Severity S2 / S3 / **S4**	14 / 24 / **4**	3 / 18 / 0	4 / 5 / **2**	7 / 4 / **3**

a badly conceived choreography, and divergence between excellent protocol metrics and unhappy users indicts G, routing to re-drafting with the human, not to patching agents.

5.2 Consequence-graded violations (S0–S4)

Raw "violation = deviation from our protocol" is circular—baselines that never saw the protocol fail by construction. We therefore (i) align observed message labels semantically before judging, then (ii) grade each surviving deviation by *consequence* against the partial order that has reasons (data and authority dependencies), annotated with irreversibility: **S0** benign reorder/dialect (not counted); **S1** waste (duplicates, polling; cost only); **S2** skipped obligation / broken order; **S3** never terminated; **S4** *irreversible act before its authorization* (disaster class). The taxonomy is itself validated: across every arm of every run, $P(\text{goal failure} \mid \text{attempt had S2+}) = 100\%$, while benign deviations did not predict failure—the grader measures harm, not difference.

5.3 Arms and fairness

All arms share the intent, role descriptions, model, step and retry budgets; the canonical validated type judges every arm (agents in A never see it). **A**: intent only. **B**: A + the full validated global type pasted as text. **C**: A + only the role's projected local contract (observer monitor). **C-min**: C with a one-line-per-transition contract. **C+**: C plus the *enforcement gate* (and, where stated, the scheduler — the full STJP stack). Experiments were pre-registered: predictions written before running, graded after (§6.6).

Table 4: Identical contracts; the only delta is the gate (+ liveness nudge). gpt-5.4, n=5.

Metric	C observer	C+ gate
GCR (strict, same attempt)	80%	**100%**
Attempts per trial	1.40	**1.00**
Delivered violations	6 / 63 events	**0 / 53**
Stalled attempts (S3)	3	**0**
Tokens per trial	114.9k	**80.9k** (−30%)
Seconds per trial	170.3	**122.1** (−28%)
LLM calls per trial	54.2	**37.6** (−31%)

Table 5: All five arms, one model, one run. "Harmful attempts" = attempts containing S2+/S4.

Metric	A intent	B global	C local	C-min	C+ gate
GCR (strict)	0%	**100%**	60%	50%	**100%**
Attempts per trial	3.00	1.00	1.90	2.00	**1.00**
Delivered violations	180/180	26/100	16/151	15/153	**0/105**
S2 / S3 / **S4**	29 / 1 / **22**	0 / 0 / 0	0 / 13 / 0	0 / 15 / 0	**0 / 0 / 0**
Harmful attempts	**97%**	0%	0%	0%	**0%**
Tokens per trial	21.7k	24.4k	154.1k	84.0k	79.5k
Tokens per success	∞	**24.4k**	96.8k	44.7k	79.5k
Seconds per trial	109	**51**	230	245	121

6 Experiments and Results

6.1 The protocol-information ladder (finance, gpt-4o, n=10)

Each step of the ladder is earned by a different mechanism (Table 3). *Validation* earns the first: agents given an *unsafe* draft completed 20%; the validated text arm, 40%. *Projection + monitoring* earns the second: 40%→80%. A's fair indictment, after severity alignment, is not "184 raw violations" (134 were dialect) but *filed the report before approval/audit four times in thirty attempts*. And C is not spotless: its two S4s were *protocol-legal but value-wrong branch choices* (taking the `Standard` branch on >$50k revenue)—exactly the hole classical MPST cannot express and choice guards close (§3); with guards compiled into contracts and monitors, this class becomes detectable (observer) and then non-completable (gate), verified by offline replay in both directions with no false positives on unevaluable guards.

6.2 Enforcement: observer vs. gate (finance, gpt-5.4, n=5, branch-balanced)

The mechanism is visible in the traces (Table 4): in both arms *TaxVerifier* repeatedly attempted to send `Approval` early. Observed, the premature send was *delivered* four times and once cascaded the session off-protocol, failing the trial; gated, the identical mistake was attempted twice, rejected pre-delivery both times, the role re-prompted—and every trial completed on the first attempt. Enforcement *paid for itself*: rejecting a wrong send is cheaper than delivering it and retrying (−30% tokens, −28% time at equal cost-per-success).

6.3 Grand five-arm comparison (finance, gpt-5.4, n=10)

Five findings, in order of importance (Table 5). **(1) B and C+ tie on outcome—and B is cheaper.** Both reach 100% with zero harm; B does it at 24.4k tokens per success vs. 79.5k. They differ in *kind*: B relies on the model *choosing* to follow pasted text (no mechanism); C+ blocks wrong messages mechanically (it intercepted five premature `Approval` sends this run). On this model and case, enforcement was sufficient but not *necessary* for the outcome; this benchmark does not show C+ beating B—it shows the *validated protocol*, present in both, doing the safety work. **(2) A stronger model makes intent-only agents more dangerous, not safer.** gpt-5.4's A-arm acted confidently and fast: 22 unauthorized irreversible acts in 30 attempts, 97% harmful, 0% completion (vs. 4 disasters under gpt-4o).

Capability amplifies the coordination gap. **(3) B's 100% is real but unguaranteed.** The same arm scored 40% (gpt-4o), 0% (gpt-4o live) and 50%-with-stalls (banking); its good behavior is a property of the model and the protocol's shape, not of the system, and it carries no mechanism preventing the A-arm failure mode. **(4) Observer projection underperformed—from liveness, not contracts.** All 13/15 failures in C/C-min were S3 stalls; C+ uses *identical contracts* plus the gate and scheduling nudge: 60%→100%. The projection must *drive* the runtime, not just judge it. **(5) Guarantees cost** $\approx 3\times$ **tokens-per-success over B** on this run—the honest price of "cannot misbehave" vs. "happened not to misbehave"—which motivates the next experiment.

6.4 Scheduler ablation: winning the cost back ($9\times$)

On a shared decentralized runtime (finance, gpt-5.4, n=10; run 2026-07-02), protocol-as-text costs 120k tokens per delivered goal at 41.8 calls/trial, because every role re-reads the whole protocol and idle roles are polled every round. Lean contract + gate: 38k at 34.0 calls. Full STJP (adding the EFSM scheduler): **13.3k tokens per goal, 11.4 calls, 32s/trial**—$9\times$ cheaper than B and -73% calls, at identical 100% GCR and zero disasters across all three (Fig. 2d). Tokens-per-call stayed flat (1.12k vs. 1.07k), so the entire saving is *fewer calls*: the mechanism, not verbosity effects. The gate rejected 10–12 off-contract send attempts per trial even in succeeding runs—enforcement measured as work done, not decoration.

6.5 Banking: S4 in dollars; and the statically caught deadlock

On the live-drafted 5-role banking pipeline (gpt-4o, n=2), intent-only agents *moved money before authorization* twice in six attempts, with nothing in their own stack flagging it; typed arms reached 100% GCR with zero S4 (and their eight raw monitor flags collapsed under grading to a single harmless S2). Separately, a two-role trading protocol seeded with the classic circular wait (Desk waits for Executor's ready; Executor waits for Desk's request) stalls every bare run forever; Scribble rejects the global type *before any agent runs*, at zero token cost, and the repaired protocol executes at 100%. No runtime guardrail, judge, or benchmark can offer this: they all require the failure to occur at least once. Testing observes runs; typing quantifies over all of them. The same circular-wait pattern is not merely authored: adapting the buyer and seller agents of a real open-source negotiation benchmark [36] (MIT) and adding an escrow settlement layer reproduces the deadlock, and a bounded live run across three model tiers shows it is capability-invariant while the validated escrow-first protocol completes (full metered benchmark pending).

6.6 Pre-registered predictions, graded — including the ones we got wrong

P1 (falsified twice, differently). We predicted unenforced global text (B) would slip below 100%: under gpt-4o it did (40%); under gpt-5.4 it did not (100%, §6.3). What survives both outcomes: the gate rejected real off-contract attempts in *every* gated run—the model tries to stray; the boundary catches it—and enforcement value is a decreasing function of model strength while the danger of *no protocol at all* is an *increasing* one (22 vs. 4 disasters). **P2 (confirmed beyond margin).** Scheduling saves $\geq$60% tokens vs. lean+gate: measured -65% tokens, -66% calls. **P3 (half-confirmed).** Projection alone does not beat global text; projection *with enforcement and scheduling* dominates it on the same runtime. **P4 (deferred).** Orchestrator-cost comparison was inconclusive (infrastructure issues on the group-chat arm). **P5 (confirmed).** Scheduling cuts calls without inflating tokens per call. We report these grades because a verification paper that hides its own falsified predictions has misunderstood its subject.

6.7 Two systems lessons

Liveness cannot be enforced by observation. The dominant failure of one gate run was inaction: a role whose whole contract is one SEND answered WAIT repeatedly; a monitor judges events, and a WAIT emits none. A contract can make every action that happens correct; it cannot, by observation alone, make an action happen. The fix is structural—the projection already contains the schedule (S5). **Tone is part of the contract surface.** An imperative liveness nudge ("WAIT is OFF-CONTRACT here; act now," with a mojibaked em-dash) made gpt-5.4 *refuse outright* in both nudged roles, stalling the arm at zero events; the same content rephrased descriptively ("Protocol status: ... the available action at this state is SEND `ExpenseData`...") was followed 5/5 first-attempt. Runtime governance text must be neutral, ASCII-safe, and informative; stronger models police the difference.

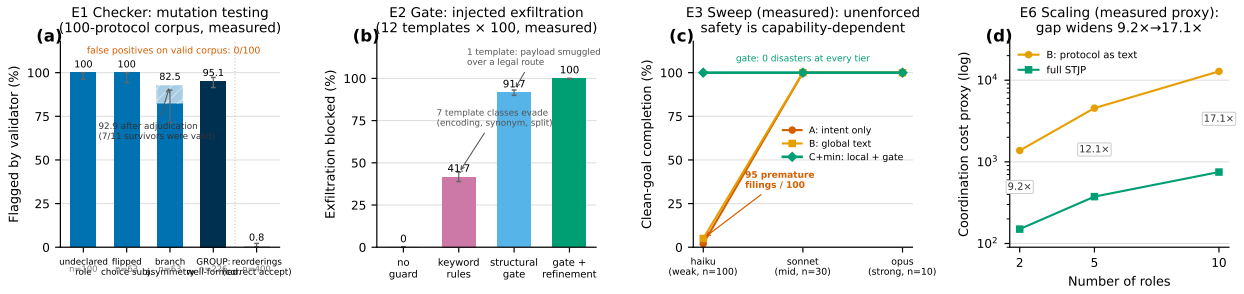

Figure 3: Validation suite. (a) E1, measured: detection of seeded defects over a 100-protocol corpus. Well-formedness defects (undeclared role, choice-knowledge violations) are what the checker must catch: 95.1% group detection, *zero* false positives on the valid corpus. Reorderings of acyclic protocols mostly yield *another valid protocol*; flagging only 0.8% of them is correct acceptance, not a miss—the greyed bar is a feature. (b) E2, measured: 1,200 injected exfiltration attempts per condition. Keyword rules are evaded by seven template classes (encodings, synonyms, splitting); the structural gate is disguise-proof for every illegal route but is blind to a payload smuggled inside a legitimately-labelled message to a legal recipient (one template, 8.3%); adding value-refinement guards closes exactly that hole: 100%. The layering is the finding. (c) E3, measured: clean-goal completion across three Claude tiers on `revenue_audit` (haiku n=100, sonnet n=30, opus n=10). Observe-arm safety and cleanliness are capability-dependent (95 premature filings at the weak tier; zero above); the gated arm is flat at 100% with zero disasters at every tier. (d) E6, measured structural cost proxy (contract size × scheduled polls): the global-text-to-STJP ratio grows from 9.2× at two roles to 17.1× at ten.

7 The Validation Suite at n=100

The pilot evidence of §6 was concept-scale ($n \leq 10$, one model family, friendly agents, an untested checker). We therefore designed a component-isolating suite—one experiment per component, each pre-registered before running—and executed it at n=100; the E3 capability sweep is now measured across three Claude tiers on both tasks. Everything below is **measured** (repository, with a per-number data map), fully terminal (0 abandoned trials after the forensic audit), and its total compute cost was under $100.

E0 — Testing the testers first. A 40-trace verdict corpus with hand-derived expected verdicts (clean traces, every severity class, concurrent interleavings on independent channels, unevaluable-guard silence, the `Approval(False)` edge case). Monitor and grader score 40/40—and, instructively, three of the *hand derivations* were wrong and corrected against the semantics; the instruments were right. All downstream experiments inherit this validated referee. An integration stress harness additionally exercises all five pipeline stages 100 times ($2105/2110 = 99.76\%$ checks passing; the five failures are a FIFO-swap mutation that yields another well-formed protocol on acyclic inputs, which Scribble *correctly* accepts).

E1 — Mutation testing the checker (Fig. 3a). Seven mutation operators over a 100-protocol corpus (2–10 roles, four topologies). The pre-registered prediction separated two families, and the data respected the split: *well-formedness* defects are detected at 95.1% (226 mutants; undeclared role and flipped-choice-subject at 100%, branch asymmetry—a role acting on branch knowledge it cannot have—at 82.5% raw) with 0/100 false positives; adjudicating the eleven branch-asymmetry survivors found seven to be *accidentally valid* mutants (correct acceptances) and four to share one reproducible genuine gap—the deletion empties a branch, projection silently drops it, and an idle role is left unconditionally awaiting a message the chooser may never send, a knowledge-of-choice deadlock the checker misses—re-scoping detection to real defects at 92.9% and converting a bare percentage into a named limitation; *reordering* operators (400 mutants) are flagged at only 0.8%—correctly, because reordering an acyclic protocol usually produces a different but valid protocol. Detecting *intent* drift under valid reorderings is the E5 scorer's job, not the type checker's; conflating the two is how benchmarks overstate their tools. The evaluation standard here is deliberately the two-sided *preciseness* discipline of Chen et al. [6, 7]—soundness (nothing bad accepted) and completeness (nothing good rejected) measured together—transplanted from characterizing a subtyping relation to auditing a checker implementation.

E2 — Adversarial exfiltration (Fig. 3b). Twelve prompt-injection templates × 100 trials against a finance protocol with a confidential payload and an ExternalAuditor role that legally receives only the public summary. No guard: 0% blocked. An honestly tuned keyword-and-role rule list: 41.7%, evaded by encoding, synonyms, typos, split pay-

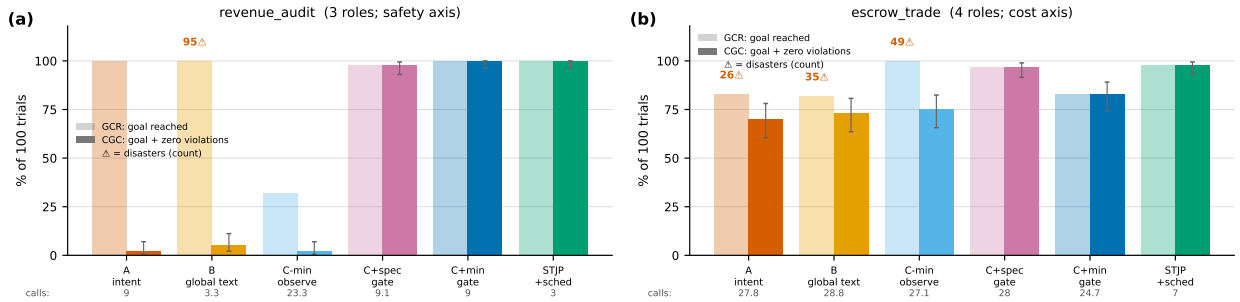

Figure 4: The six-arm ladder at n=100 (measured; 1,200 trials total, cheap subagent roles, cost in LLM calls). Light bars: goal completion (GCR); solid: clean-goal completion (CGC, goal reached with zero violations); red counts: trials containing an unauthorized irreversible act; bottom row: mean calls per trial. (a) `revenue_audit` (file only after approval): unenforced global text reaches the goal in 100/100 trials—and files before approval in 95 of them; the observe-mode local-contract arm collapses to 32% on pure liveness (an agent re-sending into silence); every enforcing arm shows zero disasters *by construction*. (b) `escrow_trade`: violations invisible at n=10 surface in every observe arm (26–49); after 18 abandoned-but-healthy C+spec trials were driven to termination the gated arms tie STJP on liveness, and the scheduler delivers the cost win: 7.0 calls/trial vs. 24.7–28.8. Post-audit final numbers; 0 non-terminal trials.

loads, and innocuous relabeling. The structural gate: 91.7%—it blocks every attempt to use an illegal (label, recipient) edge regardless of disguise, because legality is checked against the type, not the wording; the surviving 8.3% is exactly one template that smuggles the payload *inside* a legitimately-labelled message on a legal route, which structure cannot see. Gate plus value-refinement guards: 100%. Each mechanism catches precisely the class the theory says it catches.

The six-arm ladder at n=100 (Fig. 4). Two new tasks—`revenue_audit` (3 roles; the safety axis) and `escrow_trade` (4 roles, a trade settled through an escrow, i.e. a neutral third party that holds funds until both sides deliver; the cost axis)—replicate the finance ladder without Foundry, with cheap subagent role-players and n=100 per arm (1,200 trials). The six arms turn three independent knobs: *what the agent sees* (intent / global text / local projection), *whether the monitor enforces* (observe vs. gate—the same program in two modes), and *who gets polled* (all roles every round vs. only the enabled sender). Three results settle questions the pilot left open. **(i) The B tie collapses.** In §6.6, unenforced global text tied the gated arm on outcome, and we declined to claim enforcement was necessary. At n=100 under concurrent polling, arm B reaches the goal in 100% of `revenue_audit` trials while committing the forbidden act—filing before approval—in *95 of 100*. Trace forensics make the mechanism exact: in 60 of those trials the Auditor *approved without ever receiving the revenue figure* and the Filer filed in the same first round; in 35 more all three messages fired in round one; only 5 trials serialized. The filed-at-round histogram is binary: B files at round 1 in 95/100 trials; intent-only files at round 3, after approval, in 99/99 filed trials—handed the whole protocol under concurrent polling, the weak model races it; denied the protocol, it fumbles slowly into a safe order by accident. GCR conceals this entirely; CGC (goal *and* zero violations) reads 5%. Prose compliance does not survive concurrency. **(ii) Liveness, again, structurally.** The observe-mode local-contract arm completes only 32% of `revenue_audit` trials—all 68 stalls show the same pattern, an agent re-sending `Revenue` into an unenforced wait—reproducing at scale the lesson that contracts constrain what happens but cannot make anything happen; the projection must drive the runtime. **(iii) Safest and cheapest, simultaneously.** Every enforcing arm shows zero disasters at n=100 in both tasks—not zero-so-far but zero *by construction*, since the monitor in enforcing mode rejects the disallowed send before delivery—and full STJP is also the cheapest arm (3.0 and 7.0 calls/trial vs. 3.3–28.8), because the scheduler never spends a call on a role whose turn it structurally is not. On `escrow_trade`, once eighteen abandoned-but-healthy C+spec trials were driven to termination (see the audit below), the gated arms tie STJP on liveness (97–98%, all at 0 disasters); STJP's escrow advantage is then *purely* the 4× cost edge—an honest decomposition we prefer to an inflated gap. **(iv) Reconciliation: no unenforced arm is safe across conditions.** Holding the finance run and the ladder side by side, the failing arm *moves*: intent-only was the villain under a strong confident model (22 disasters, finance), unenforced global text under concurrent polling (95 disasters, `revenue_audit` — trace-verified: `Filer→Analyst:Filed` in round 1 with no `Approval` preceding it, because poll-all hands the Filer its first turn before approval can exist), and the observe-mode local contract on liveness (31%) or violation volume (49, `escrow_trade`). Which unenforced configuration fails is a function of model, task, and scheduling that nobody can predict in advance—and the measured capability sweep (Fig. 3c) closes the model axis: on identical

Table 6: Cost to *clean* goal (mean calls/trial ÷ CGC), n=100 per arm. GCR-based cost rankings reverse once violated runs stop counting as successes.

Task	A intent	B global text	C-min obs.	C+spec	C+min	STJP
`revenue_audit`	450	66	1165	9.3	9.0	**3.0**
`escrow_trade`	39.7	39.5	36.1	28.9	29.8	**7.1**

Table 7: E4, measured: reliability at n=100 (infrastructure trials; Wilson 95% intervals). Narrowing the CI is what n=100 buys: pass^{10} at the CI floor rises 17.6×.

Arm	succ/n	95% CI	pass^{10}@LB
unchecked, n=10	0/10	[0.0, 27.8]%	0
STJP, n=10	10/10	[72.2, 100]%	0.039
unchecked, n=100	0/100	[0.0, 3.7]%	0
STJP, n=100	100/100	[96.3, 100]%	**0.686**

Table 8: E5: translation-fidelity instrument, measured; LLM-dependent measures pending.

Measure	Result
EFSM-equivalence scorer accuracy	300/300
identical / reformatted / mutant	100% each
First draft passes validator	PENDING
Repair rounds to validity	PENDING
Guard sidecar co-emission	PENDING

protocol text, B's premature filings go 95→0→0 across the haiku/sonnet/opus tiers, intent-only's cleanliness goes 2%→100%, and the escrow sweep mirrors both (observe-arm disasters 26–35→0). The enforcing arms are the only *invariant* of the whole grid—zero disasters in every task, model tier, and scheduler tested, with zero gate rejections even needed at the stronger tiers. Notably, the strong model's *safe* B costs 9.0 calls/trial against reckless haiku's 3.3: the unenforced arm pays for safety in serialization, while STJP gets safety and 3.0 calls simultaneously because the scheduler neither races nor idles. Table 6 prices this: dividing mean calls by CGC, B's apparently cheap 3.3 calls/trial becomes 66 calls per *clean* goal once poisoned successes stop counting, and full STJP wins by 4–22× on the metric an operator actually pays—while being the safest arm rather than trading safety for it. In dollars (weak-tier roles at list price): a clean, safe audit costs $0.38 under STJP versus $1.12–$9.09 for the alternatives, and the entire 1,200-trial suite cost under $100.

E3 — Capability sweep (measured; Fig. 3c). Three Claude tiers (haiku n=100, sonnet n=10–30, opus n=10; one mind per trial, every trial verified against raw state) across both ladder tasks. Three findings. *Unenforced safety is capability-dependent*: B's premature filings fall 95→0→0 on identical protocol text, and escrow's observe-arm disasters fall 26–35→0. *Unenforced cleanliness is capability-dependent*: intent-only rises from 2% to 100% CGC as duplicate-send waste vanishes—a stronger model self-organizes without being shown the protocol. *Enforcement is capability-invariant*: the gated arm is 100% CGC, zero disasters, zero gate rejections, at every tier. As capability rises the unenforced arms approach the gated arm; read naively, that says strong models need no enforcement—read correctly, the closing gap is an *average* while the gate's zero is a *guarantee*: production does not get to assume the strongest model, and even strong models carry a failure tail that enforcement makes exactly zero. The one remaining E3 gap is vendor diversity (a non-Claude frontier point), an access limitation we report rather than fake. The shape of this debate is not new: every type system in history has faced "skilled programmers don't need it," and the answer adopted by every safety-critical industry is that averages improve with skill while tails do not vanish, and criticality is priced on the tail. The designed probe for the strong-model tail already exists in this suite: E2, where the violation is produced by the attack surface rather than by model weakness; running E2 with frontier-tier agents is the experiment that makes the tail visible at the top of the capability curve, and it is first on our post-submission list.

E4 — Reliability (Table 7). The n=100 runs use a contract-following strategy in place of a live model: they certify the *infrastructure* (gate rejects exactly the illegal sends, scheduler polls exactly the enabled role, monitor emits clean traces) at scale, while live-model behavior is carried by the finance runs (§6) and the subagent ladders above. Under that scoping: the unchecked arm's interval narrows to $[0, 3.7]\%$—structurally incapable, not unlucky—and the STJP arm's pass^{10} at the Wilson floor rises from 0.039 to 0.686. The n=100 deadlock replication is the same story in miniature: the unchecked configuration deadlocks in 100/100 trials, always in round two, burning 800 calls; STJP completes 100/100 at the protocol-optimal 7.0 calls/trial—*fewer total calls than the arm that never finishes*.

E5 — Translation fidelity (Table 8). The deterministic half is done: the EFSM-bisimulation (a state-machine equivalence check meaning the two behave identically no matter what happens) scorer that will judge LLM-drafted protocols against expert gold (a known-correct reference protocol) classifies 300/300 pairs (identical / semantics-

preserving reformat / seeded mutant) correctly. The LLM-dependent half—first-draft validity, repair rounds, guard co-emission—remains the declared open piece of the seam and is measurement-pending; the scorer's validation is what makes those forthcoming numbers trustworthy.

Training the seam is possible, not merely a stated goal. Because the target grammar, the well-formedness validator, and the EFSM-equivalence scorer (Table 8) already exist, intent→protocol translation is a verifier-in-the-loop learning problem with a free, deterministic reward. We do not leave that observation as a promissory note: §8 reports the training program built on it—the system under training, the instruments (grammar, corpus, faithfulness panel, real-skills mining) measured before any weights move, and the preregistered gates whose GPU outcomes are this paper's declared next step and fill the pending E5 cells above through the shared results template.

E6 / E7 — Scaling and portability. The roles sweep (Fig. 3d) confirms the pre-registered shape on a structural proxy: global-text coordination cost grows super-linearly (every role re-reads a growing protocol under poll-all), STJP near-linearly (each role sees only its slice; one enabled poll per step), with the ratio widening from $9.2\times$ to $17.1\times$ between 2 and 10 roles; token-metered curves are pending. For portability, the standalone generated monitor agrees with the in-process monitor on 59/59 testable canonical traces (41 choice-only protocols yield no meaningful canonical trace; the claim is scoped to testable ones), and a LangGraph adapter executing the ladder protocols as StateGraphs agrees with the native monitor on 14/14 clean-and-injected checks, consistent with T4's placement of enforcement at message boundaries; the skill-runtime third harness is pending.

Real skills, run live: nine cases and a livelock (Table 9). The mining result above measures what authors *write down*; live execution measures what the written-down artifacts *do*. Nine multi-agent cases were assembled from real, unmodified public sources—Anthropic's skills repository, GitHub's `awesome-copilot` collection, and the OpenAI Agents SDK, LangGraph, AutoGen, CrewAI, and AgenticPay example corpora, every source file deep-linked with license provenance—and each was executed through the trial engine under the three canonical conditions: the found files as downloaded (*no plan*), the corrected coordination plan pasted into every skill as prose (*plan as text*), and the full compiled stack (contract + gate + scheduler), with every role played by an independently spawned subagent. Four findings. **(i) Found files never coordinate reliably.** Seven of nine cases complete 0/10 under the unmodified files—deadlock or stall, every trial—and the remaining two complete only as a model-dependent coin flip whose direction *reverses* between Claude tiers: on 120 two-model trials, Haiku ships the announcement 10/10 but the code change 0/10, and Sonnet exactly flips. Swapping in a stronger model does not supply the missing structure; it relocates the failure—the E3 capability lesson, replicated on found artifacts. **(ii) Plan-as-text completes without complying.** Where the textual plan reaches the goal it does so with the violation load the ladder predicts: 10/10 double seat-writes on the airline case, 10/10 double charges on the booking saga, 120–220 rule-breaking messages per run on the multi-round cases. **(iii) Looping protocols expose a failure mode the acyclic suite could not: livelock.** The corrected code-review case (`pr_review_merge`: a `rec` loop with nested `choice` at two concurrent reviewers, ending in a two-approval join at the Merger) is the first live run of a recursive protocol through the engine, and on it the plan-as-text arm does not merely fail—it enters a *stable livelock*: all 10 trials burn the entire 16-round budget in motion, emitting 530 rule-breaking messages and ∼42k tokens per trial while merging nothing, at $3.6\times$ the token cost and $5.3\times$ the LLM calls of the STJP arm that merges 10/10 with zero violations in 7 rounds. Its sibling (`doc_coauthor_ship`, the reader-test refinement loop from Anthropic's `doc-coauthoring` skill) completes under text but with 220 violations, while STJP is clean, loops the reader test more than once before shipping, and is still $\sim 40\%$ cheaper. Deadlock is silence and livelock is motion without progress; the same static object rules out both, because both are properties of the protocol, not of any message. **(iv) The checker earns its keep on the way in.** The corrected review protocol was itself debugged statically: two Scribble rejections ("inconsistent external choice subjects"—a role that could not tell which branch the team had taken; "role progress violation"—a role that could go rounds without a message) preceded validation, the §3 diagnostics firing on a realistic review loop before any agent ran. Two honesty notes: the earlier casts of two of these cases had misread their sources (the `address-comments` agent presupposes an *existing* PR; the brand-guidelines skill contains no approval language), and the corrected cases move the loop and the gate to where the source files actually put them; and the runs are $n{=}10$ per arm with round-batched role-play, so trials of one role within a round share a model context and are not fully independent.

Pre-registration, graded; and the audit that changed a number. All eight v2 predictions were registered before running and graded after: E0, E1 (with the reordering-acceptance clause), E2 (with the predicted legal-route smuggle), E4, E5-scorer, E6-proxy, and E7-agreement CONFIRMED; E3, registered pending, is now measured with its capability clause confirmed and vendor diversity still open. A post-hoc forensic audit of the full dataset then re-verified every suspicious pattern: B's low-diversity replies were adjudicated genuine on three independent behavioral signals (three

Table 9: Real-skills live runs: nine cases built from unmodified public agent/skill files, three arms each. "H"/"S" = Claude Haiku/Sonnet tiers in the two-model sweep (120 trials); *viol.* = rule-breaking messages. The `pr_review_merge` text arm is the measured livelock: 0/10 at $3.6\times$ the cost of the succeeding STJP arm.

Case	Source	No plan	Plan as text	Full STJP
`airline_seat`	OpenAI Agents SDK	0/10 deadlock	10/10, 10 dbl. writes	**10/10, 0, cheapest**
`booking_saga`	LangGraph	0/10 stall	10/10, 10 dbl. charges	**10/10, 0**
`code_execution`	AutoGen	0/10 deadlock	10/10	**10/10, cheapest**
`content_pipeline`	CrewAI (pattern)	0/10 stall	10/10	**10/10, cheapest**
`doc_pipeline`	anthropics/skills	flip: H 10/10, S 0/10	20/20, 120 viol.	**20/20 both, 0**
`pr_merge`	awesome-copilot	flip: H 0/10, S 10/10	20/20, 120 viol.	**20/20 both, 0**
`agenticpay_settlement`	AgenticPay	deadlock, all 3 tiers	—	**100%, all 3 tiers**
`pr_review_merge`	awesome-copilot	0/10 deadlock (r.2)	**0/10 livelock**, 530 viol.	**10/10, 0,** $3.6\times$ **cheaper**
`doc_coauthor_ship`	anthropics/skills	0/10, budget exhausted	10/10, 220 viol.	**10/10, 0,** $\sim$**40% cheaper**

distinct trace shapes, including a five-trial serialized branch no script would produce); intent-only's zero disasters trace to accidental serialization, its 591 violations all named duplicate-send waste; and the audit surfaced one material bug the checklist had not targeted—22 of 1,200 trials were abandoned mid-play rather than failed, 18 of them depressing one gated arm's reported liveness to 79% against a true $\sim$97%. All 22 were driven to termination and every table regenerated; the 19 *genuine* gated-arm deadlocks were left standing, because replaying real failures until they pass is p-hacking. We also log the integrity incidents scale surfaced: subagent role-players caught writing auto-responder scripts instead of reasoning per poll (detected by a `malformed==calls` signature and self-confession, discarded and replayed), a round-numbering bug that could mask disasters (fixed with causal detection), and a false-positive disaster check on enforcing arms (fixed mid-analysis); the final audit verified all 1,200 ladder trials against raw state files with zero remaining fraud and zero non-terminal trials. We report these because a verification paper that does not verify its own experiments has misunderstood its subject.

8 Training the Intent-to-Protocol Translation Step, Realized

The one part of STJP's own pipeline it cannot type-check is its own front door: intent→Scribble passes through an LLM, and a *valid-yet-unintended* protocol survives the validator (§10). §7 argued this translation step (the seam) is trainable; this section reports the program that realizes the argument—a system under training, a set of instruments measured before any weights move, and a preregistered set of go/no-go gates. Consistent with the rest of this paper, everything below is either *measured today* or *explicitly pending*: the GPU training runs are the declared next step, and their numbers enter through a results template (§8.4) rather than by rewriting a table.

8.1 Two axes, treated differently

Translation quality has two axes with opposite epistemics. *Validity*—well-formedness, projectability, guarded recursion, guard satisfiability—is decided by the real Scribble-java toolchain, a total, counterexample-producing oracle: a draft is well-formed or it is not, and when it is not the checker returns the offending role or cycle. *Faithfulness*—does G encode what the user meant—is not machine-decidable and is the residual judged surface (§5). The program treats them accordingly. Validity is a free, deterministic reward that can drive training directly; faithfulness is measured by a memoryless judge panel that must itself pass a calibration gate before it may reward or gate anything, and until it does the reward is verifier-only. This is the paper's judge-free discipline carried into training: shrink the judged surface to what is genuinely semantic, and never let an uncalibrated judge touch a number.

8.2 The system under training and its verifier reward stack

Two artifacts share one open-weights base (a 7B code model, LoRA—Low-Rank Adaptation, a lightweight fine-tuning technique that trains a small add-on set of weights instead of the whole model—heads differing): a *translator* T (intent → `.scr` + `.refn` guard sidecar) and a *repairer* R (intent, broken G, validator counterexample → fixed G); at inference they run the same draft→validate→repair loop the live drafting already exhibited (four rejected drafts before

the fifth passed, §4.1). The reward is a verifier stack, not a preference model: grammar-constrained decoding removes syntactic invalidity by construction; the validator contributes a pass reward; a *cheap* canonical-EFSM equivalence proxy (roles matched, transition-label multiset overlap, branch/recursion skeleton) contributes graded equivalence-to-gold, with full bisimulation reserved for evaluation because we measured it at 10–$40\times$ the cost of a validation—too slow for a rollout loop; guard co-emission is rewarded; and the faithfulness term stays exactly zero until the panel's calibration gate passes. The training recipe is deliberately standard—verifier-filtered sampling into supervised fine-tuning (expert iteration [38]), then group-relative RL from verifiable rewards [39] with the post-R1 fixes for length bias and entropy collapse [40, 41, 42], and back-translation from a generator we own [43, 44]—because the novelty is the target and its total checker, not the optimizer. One caveat we take seriously and preregister around: constrained decoding can suppress semantic quality, not merely syntax [45, 46], so grammar-on vs. grammar-off is a primary two-sided test (H1 below), not an assumption.

8.3 The instruments, measured before any training

The four instruments the program needs already exist and are measured against the real toolchain (Table 10). The grammar admits exactly the corpus and nothing malformed. The corpus builder expands and EFSM-deduplicates families with a canonical signature whose verdicts agree with the pairwise equivalence checker on every tested pair, and splits them by structural family so no paraphrase or mutant straddles the train/test line. The faithfulness panel, on its first live run, did the two things it was built to do: it rejected a swapped intent/protocol canary (a planted check item with a known correct answer, mixed in to verify the panel itself is working) at 0.99, and—on `trade_deadlock`, whose intent is deliberately deadlock-shaped and whose gold protocol implements the deadlock-*free* linearization—both forward seats accepted the protocol as faithful (0.88, 0.82) while the blind back-translation seat, which never sees the intent, reconstructed a strictly linear happy-path and scored 0.25 against the original. The class structure caught a confirmation bias the forward seats shared: the protocol *repairs* the intent rather than encoding it, a policy question that escalates to the human gate, not a vote. Finally, the real-skills mining run is a null result reported with its controls and its follow-up. The deterministic (no-LLM) path yielded zero of 13 mined teams surviving compaction into a coordinating protocol; a synthetically authored team whose skills state their interaction structure explicitly passes the same pipeline end to end, locating that failure in the inputs, not the pipeline. We then ran the disambiguating follow-up—careful model-read extraction under an evidence-only discipline (every recovered interaction must quote the skill text) over the same 13 teams—to separate *structure absent* from *structure implicit in prose*. Reading harder recovers some structure but little: 3 of 13 teams yield a Scribble-valid protocol and a 4th surfaces a genuine deadlock the compatibility check correctly rejects, while 7 of 13 have nothing to surface. Decisively, the recoveries come from this project's own worked examples and one orchestrator file, whereas the teams built from unmodified upstream GitHub files—the only directly comparable real-world inputs—recover *zero*: ordinary skill authors do not write down interaction structure even when a human-discipline reader tries hard to find it. A caveat the follow-up itself exposed—the harvester grouped files into "teams" without first checking that the underlying task *requires* multi-party coordination (six of the thirteen did not)—became the next experiment: we built that task-level filter and re-ran the harvest at scale. Across seven public sources (923 harvested artifacts, 52 from an unlicensed repository quarantined; 110 candidate teams), the filter identifies **29 teams whose tasks genuinely require multi-party coordination**, every verdict carrying verbatim evidence quotes from the source text. On those 29: the deterministic path still recovers zero; careful evidence-only reading, applied to 16 of the 29, recovers a checker-valid protocol for 7 and surfaces genuine role incompatibilities in 3. The sharpest regularity is *where* recovered structure lives: per-agent skill files across three independent catalogs describe coordination-requiring tasks in only 2–7% of groupings, while orchestration configurations state the flow essentially whenever they are present at all—coordination, when written down, lives in the orchestrator layer, not in the per-agent artifact, which is precisely the layer this compiler formalizes. Two scope notes keep this honest: the measured population is what authors *write down*, not what deployed systems *need* (a low rate in artifacts implies nothing about demand for coordination); and mined data still yields too few `test-real` items to stand alone: of the recovered protocols, four trace to permissively licensed upstream repositories and are releasable, while a fifth traces to a repository that carries no license and is therefore withheld [35]. The human-read baseline is packaged and pending.

Table 10: The seam instruments, measured before any training, against the real Scribble-java toolchain. Corpus and repair counts are from bounded runs whose saturation curves project to the plan's targets; the reachable numbers (and their continue-commands) are reported rather than padded.

Instrument	Measured today	Role
Scribble grammar (GCD)	100% corpus round-trip (113/113 non-skip `.scr`; the 5 skips are rejected LLM drafts, correctly unparseable); 1,000/1,000 sampled protocols parse; 0 parse-level rejections under real Scribble	syntactic reward = 0 by construction
Protocol corpus	671 EFSM-deduped families (bounded run; ≥5k reachable); canonical signature vs. pairwise checker 200/200 agreement; 860 repair tuples + 1,746 calibration near-misses; family splits leakage-check GREEN (599/76/76)	SFT/GRPO data; leakage-proof splits
Faithfulness panel	14 seats, live; swapped-pair canary rejected at 0.99; blind back-translation caught a shared forward-seat confirmation bias (`trade_deadlock`: fwd 0.88/0.82 vs. back 0.25)	faithfulness metric (gated, §8.4)
Real-skills mining	923 artifacts, 110 candidate teams, 29 coordination-requiring (task-level filter, evidence-quoted); deterministic path 0/29; careful reading recovers 7/16 assessed — stated coordination lives in orchestration configs, not agent files	`test-real` source; in-the-wild evidence

8.4 Preregistered gates, and the results template

The training outcomes are governed by six go/no-go predictions written before any run, in the same discipline as the n=100 suite (§6.6). **H1** runs grammar-constrained decoding on vs. off as a primary two-sided test at both training phases: if constraint costs more than two points of semantic validity or graded equivalence [45], it is demoted to data-generation filtering. **H2** (a measurement, not a gate) predicts best-of-n validated sampling lifts validity@k well above validity@1 and fills the pending E5 cells. **H3–H4** gate the SFT and GRPO checkpoints on validity, graded equivalence, and repair-rounds, with H4 written headroom-aware so a successful H3 (validity near ceiling) switches the primary H4 metric to graded equivalence rather than demanding an arithmetically impossible validity gain. **H5** is the panel calibration gate, and its statistics are honest about power: certifying an 85% agreement threshold needs on the order of 370 human-labeled items, so the gate binds on a Wilson 95% lower bound ≥ 0.80 over $n \geq 200$ items rather than an 85% point estimate, and on non-circular strata (mined and human-audited, not the back-translation winners the judging family itself selected). **H6** reports transfer to human-written intents as a bootstrap point estimate with a confidence interval, because at n=150–300 the gap-of-gaps is underpowered as a binary test—the interval width is reported, not hidden. If a gate misses, the program degrades gracefully (SFT ships without GRPO; the panel stays advisory forever) because the verifier rewards never depended on the judge.

Tables 11 and 12 are the template. Every cell is a macro from `seam_results.tex` that currently renders PENDING and is replaced, post-GPU, by editing that one file (`TEMPLATE_HOWTO.md` maps each macro to its harness output field); the table structure never changes, so a compiled cell reads either a measured number or an honest PENDING—exactly as the E5 and E3-vendor cells elsewhere in this paper do.

Positioning and verified novelty. The recipe's lineage is explicit—RL from verifiable rewards [39, 40, 41, 42], expert iteration [38], back-translation and verifier-checked autoformalization [43, 44], and independent judge panels over a single judge [47] with the 2026 evidence that panels carry far fewer effective votes than seats [48] and need a robust (geometric-median) aggregator [49], both of which this panel adopts. The task itself, however, appears to be new: nine query variants across the MPST/Scribble/choreography vocabulary crossed with LLM/NL-generation phrasing returned no prior system translating natural-language intent into multiparty session types. Two near-misses are named to pre-empt the comparison: ZipperGen [17] shares the projection-to-deadlock-free-endpoints philosophy but has no natural-language front end (it is a programmer-authored MSC DSL), and Liu et al. [50] shares the two-stage NL→formal-spec shape but targets network I/O grammars for test generation, with no global type, projection, or well-formedness dimension. Neither anticipates a global-type-with-projection target drafted from intent.

Table 11: T0 prompt-era baselines (no weights): validated best-of-n over three API tiers. Template; all cells are macros.

Metric	Haiku	Sonnet	Opus
validity@1	PENDING	PENDING	PENDING
validity@10	PENDING	PENDING	PENDING
repair-rounds	PENDING	PENDING	PENDING
$-to-accepted	PENDING	PENDING	PENDING

Table 12: Trained systems and panel calibration. Template; all cells are macros, PENDING until the GPU runs land.

Measure	Result
SFT-7B validity@1 (test-syn)	PENDING
SFT-7B bisim@1 (equiv. to gold)	PENDING
SFT-7B $-to-accepted	PENDING
GRPO Δ validity@1 vs. SFT	PENDING
GRPO Δ graded-equiv.@1	PENDING
GRPO Δ repair-rounds	PENDING
Panel AUC (gold vs. mutant)	PENDING
Panel human-agree (Wilson LB)	PENDING
Panel swapped-pair rejection	PENDING
Panel effective votes	PENDING
Transfer gap (Δ of gaps)	PENDING

Table 13: Scorecard for the three extensions (E8–E10). "Shipped" is the full STJP of §7.

Extension (theory)	Shipped STJP	With extension	Verdict
E8 session ledger [5]	blind to cumulative overruns: 0/50 detected; live, 16 over-budget debits paid	50/50 caught at the exact crossing message, 0/50 FP; live, 0 paid, goal still reached	new guarantee
E9a subtype build check [6]	"lean $\leq$ projection" asserted in prose	decidable build step; catches a lean contract that drops a required send	hardening
E9b tolerant gate [6, 7]	strict gate	0/50 illegal sends admitted; 0 anticipations needed live	safe; boundary result
E10 crash handling [8]	100% limbo on crash (14/14 grid; 19/19 replays; 15/15 live)	100% typed terminal, 0 disasters; 5/5 unsafe handlers rejected statically	new checked property

9 Three Typed Extensions, Measured

The validation suite certifies what STJP ships; this section extends what it can *catch*, *build-check*, and *survive*, with one prototype per session-type result: guards [4] $\rightarrow$ stateful properties [5] $\rightarrow$ lawful slack [6, 7] $\rightarrow$ typed recovery [8]. Each prototype passed a hand-derived verdict corpus (12/14/12 traces) before its benchmark ran; all predictions were pre-registered and graded, including the one we grade against ourselves. These are failure-mode coverage additions, not speedups: none moves the core cost or safety numbers of §6–§7, and each write-up names the shipped system's blindness as the motivation, not a planted defect.

E8 — Stateful session invariants (the ledger). Per-message guards see one message; a budget of $10k with a per-debit limit of $5k is overrun by three $4k debits that every shipped check individually approves—a violation class the gate is blind to *by construction*. Following Chen & Honda [5], the `.refn` sidecar gains `state`/`on`/`invariant` clauses; the generated monitor carries a session store, applies updates on accept, and checks invariants before delivery (centrally at the gate in v1, deferring the per-endpoint distribution—where the asynchronous machinery of Chen & Honda [5] becomes essential—to future work). Measured: on 100 seeded traces, the shipped arm detects 0/50 overruns, the ledger flags 50/50 *at the exact crossing message* with 0/50 false positives; in live trials (a $12k procurement against the $10k budget, n=15/arm) the shipped gate paid 16 over-budget debits undetected, the ledger-observe arm flagged 15/15, and the ledger *gate* paid exactly $10,000 in every trial while still reaching the goal—the Treasurer, re-prompted with the invariant text, downsized or refused. Cost: a few sidecar clauses. Bounds: invariants are checked dynamically (not SMT-proved) and centrally.

E9 — Lawful slack (precise subtyping), a win and a boundary. Two deliverables from one theorem. *(2a)* A coinductive subtype checker (output covariance, input contravariance, send-polarity) makes lean $\leq$ projection a decidable *build step*: contract compression is now verified rather than asserted, and the checker catches a seeded lean contract that drops a required send. *(2b)* A tolerant gate implementing the one decidable, provably safe relaxation—

independent-receive output anticipation—behind a default-off flag. The mandatory control holds: 0/50 illegal sends admitted, so tolerance never widens the gate beyond the precise relation. The deadlock replay, pre-registered as uncertain ("≥5/19 rescued"), returned the more interesting answer: 16/19 deadlocks do contain a rejected send that subtyping proves type-safe (the strict gate *is* stricter than safety requires), yet all 16 share one root cause—the Buyer never sent `Deposit`; the rejected sends were other roles correctly waiting on an absent participant, so admitting them rescues nothing—and live reruns needed 0 anticipations (n=15/arm, both gates 15/15). The boundary result is worth stating plainly: on authorization-laden protocols, message orderings *encode obligations* (pay-before-ship), so the slack that subtyping licenses at the type level is precisely what one does not want to spend; lawful slack is real, and this domain is where you leave it unspent. The type theory and the deployment answer diverge, and reporting both is the point.

E10 — Typed crash handling (no session left in limbo). The audit's 22 non-terminal trials were untyped crash failures; following Viering et al. [8], a `.fail` sidecar declares per-region handlers, the validator statically checks *coverage* (every state × crashable role handled or explicitly escalated), *handler safety* (handler bodies are re-validated as mini global types through the same Scribble pipeline), and *recoverability* (every crash reaches a typed terminal); the scheduler—already the coordinator—gains a poll-timeout crash detector, and monitors swap to pre-generated handler EFSMs. Measured: the full crash-point grid (14 cells) leaves the shipped system in limbo 14/14 and STJP+CF at 14/14 typed terminals (9 degraded, e.g. `Refunded`, 5 clean aborts) with 0 disasters; all 19 real recorded deadlocks replay to typed terminals; live flaky-role trials (n=15, one role goes silent per trial) resolve 15/15. Critically, the new checker is held to the E1 preciseness standard: 5/5 seeded unsafe handlers are rejected statically—including a settlement-shortcut handler whose "recovery" would bypass authorization, exactly the class a naive recovery mechanism would smuggle in. "No session left in limbo" is now a compile-time property, and the audit finding that embarrassed us most is the one this section closes.

10 Limitations

The intent-to-protocol translation step. Intent→Scribble and contract→skill rendering pass through an LLM; the validator rejects structurally invalid drafts, but a valid-yet-unintended protocol survives (validated ≠ intended). Shipped mitigations: human endorsement of G; the outcome anchor. Underway: §8 realizes the seam as a verifier-in-the-loop learning program—validated grammar, EFSM-equivalence scorer, EFSM-deduplicated corpus, and a calibration-gated faithfulness panel, all measured before training—with best-of-n validated sampling as the training-free baseline and back-translated corpus pairs as the fine-tuning path; the GPU training outcomes are preregistered (H1–H6) and pending, entering the paper through the §8 results template. **Scope of the theorems.** T3 is per-session; interleaved sessions need compositional/hybrid extensions; dynamic role join/leave needs [12]; our experiments fix the role set. **Expressiveness.** Open-ended brainstorming resists protocolization, and forcing it into a type would be dishonest; STJP targets the coordination-critical, authorization-laden regime. Probabilistic session types are a candidate middle path. **Statistics and scope of the n=100 suite.** The live gpt-4o/gpt-5.4 finance runs remain $n \leq 10$ per cell; the n=100 suite fixes the statistical power but substitutes cheap subagent role-players (ladders) or contract-following strategies (E4) for frontier models, and plays each trial's roles from one subagent's per-role views rather than truly independent agents; the nine real-skills live runs (§7) are n=10 per arm with round-batched role-play, so same-role trials within a round share one model context and are not fully independent. The E3 sweep now spans three Claude tiers on both tasks but lacks a non-Claude frontier point (an access limitation), so vendor generality of the capability curve remains open; the tone finding (§6.7) especially needs cross-model replication. **Grader and guards.** The post-hoc severity grader is payload-blind at milestones (`Approval(False)` currently counts as the authorization milestone; the monitor-level guard closes this, the grader upgrade is pending); choice-guard authoring is manual (the drafter should co-emit the sidecar). **Extension bounds.** The crash-handling roadmap item is now implemented and measured (§9), with its own bounds: a `.fail` sidecar must be authored per protocol, and the live crash model is a role going silent. The session ledger checks invariants dynamically and centrally, not by SMT proof or per-endpoint distribution [5]; the tolerant gate is provably safe but, on obligation-encoding protocols, deliberately left off by default (§9). **Monitor correctness** is validated (theory-matching, regression tests, manual audits), not mechanized.

11 Conclusion

When skills became the programming language of multi-agent systems, the field skipped the compiler and went straight to observability. STJP retrofits the missing stage: intents compile into a protocol algebra with a forty-year pedigree; faulty choreographies are rejected before a single token is spent; and the artifacts agents actually receive—lean local skills, generated monitors, guards, a protocol-driven scheduler—are projections of a proven object rather than prose approximations of a hopeful one. A 1,200-trial validation suite replicates the shape at $n{=}100$: observing arms accumulate real violations that goal-rate metrics hide, enforcing arms show zero by construction, and the checker itself survives mutation testing with zero false positives. Nine cases built from unmodified public skill files replicate the shape in the wild and add its sharpest form: on a looping review protocol the plan-as-text arm livelocks through its entire budget at $3.6\times$ the cost of the compiled arm that ships—the failure costs more than the guarantee. The empirical shape is what compilation always buys: the same guarantees, then made cheap ($9\times$ on cost-to-goal once the projection is allowed to *drive* the runtime), plus a class of properties—deadlock-freedom, goal reachability, authorization ordering, guarded branching, recipient confidentiality—that no amount of runtime watching can provide, because they quantify over runs that never happen. Prompts move probabilities; only checks move guarantees. The open seam—natural language in, formal protocol out—is real, measurable, and, we argue, the right next benchmark for the field.

References

[1] K. Honda, N. Yoshida, M. Carbone. Multiparty asynchronous session types. *POPL* 2008; *J. ACM* 63(1), 2016.

[2] A. Scalas, N. Yoshida. Less is more: multiparty session types revisited. *POPL* 2019.

[Scribble] K. Honda, A. Mukhamedov, G. Brown, T.-C. Chen, N. Yoshida. Scribble: describing multiparty protocols. Scribble project; `scribble-java`, `nuScr`, `mpstk` toolchains.

[4] L. Bocchi, K. Honda, E. Tuosto, N. Yoshida. A theory of design-by-contract for distributed multiparty interactions. *CONCUR* 2010.

[5] T.-C. Chen, K. Honda. Specifying stateful asynchronous properties for distributed programs. *CONCUR* 2012, pp. 209–224.

[6] T.-C. Chen, M. Dezani-Ciancaglini, A. Scalas, N. Yoshida. On the preciseness of subtyping in session types. *Logical Methods in Computer Science* 13(2), 2017.

[7] T.-C. Chen, M. Dezani-Ciancaglini, N. Yoshida. On the preciseness of subtyping in session types: 10 years later. *PPDP* 2024, 2:1–2:3.

[8] M. Viering, T.-C. Chen, P. Eugster, R. Hu, L. Ziarek. A typing discipline for statically verified crash failure handling in distributed systems. *ESOP* 2018, pp. 799–826.

[9] Refinements for multiparty message-passing protocols. *ECOOP* 2024.

[10] L. Bocchi, T.-C. Chen, R. Demangeon, K. Honda, N. Yoshida. Monitoring networks through multiparty session types. *FORTE* 2013; *TCS* 2017.

[11] A. Igarashi, P. Thiemann, Y. Tsuda, V. Vasconcelos, P. Wadler. Gradual session types. *ICFP* 2017; *JFP* 2019.

[12] P.-M. Deniélou, N. Yoshida. Dynamic multirole session types. *POPL* 2011.

[13] P.-M. Deniélou, N. Yoshida. Multiparty session types meet communicating automata. *ESOP* 2012.

[14] N. Yoshida, P.-M. Deniélou, A. Bejleri, R. Hu. Parameterised multiparty session types. *FoSSaCS* 2010.

[15] F. Montesi. *Introduction to Choreographies*. Cambridge University Press, 2023.

[16] M. Carbone, F. Montesi. Deadlock-freedom-by-design: multiparty asynchronous global programming. *POPL* 2013.

[17] B. Bollig, M. Függer, T. Nowak. Provable coordination for LLM agents via message sequence charts. arXiv:2604.17612, 2026. ZipperGen; results machine-checked in Lean 4.

[18] E. Li, F. Stutz, T. Wies, D. Zufferey. Complete multiparty session type projection with automata. *CAV* 2023.

[19] E. Li, F. Stutz, T. Wies, D. Zufferey. Characterizing implementability of global protocols with infinite states and data. *Proc. ACM Program. Lang.* 9(OOPSLA1), 2025.

[20] E. Li, T. Wies. Certified implementability of global multiparty protocols. *ITP* 2025.

[21] C. Paduraru, P.-L. Bouruc, A. Stefanescu. A trace-based assurance framework for agentic AI orchestration: contracts, testing, and governance. arXiv:2603.18096, 2026.

[22] M. Kaptein et al. Runtime governance for AI agents: policies on paths. arXiv:2603.16586, 2026.

[23] H. Wang, C. Poskitt, J. Sun. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents. *ICSE* 2026; arXiv:2503.18666.

[24] Agent-C: enforcing temporal constraints for LLM agents. arXiv:2512.23738, 2025.

[25] I. Levy, B. Wiesel, S. Marreed, A. Oved, A. Yaeli, S. Shlomov. ST-WebAgentBench: evaluating safety and trustworthiness in web agents. *ICLR* 2026; arXiv:2410.06703.

[26] X. Wei et al. BeSafe-Bench: unveiling behavioral safety risks of situated agents in functional environments. arXiv:2603.25747, 2026.

[27] S. Saha et al. Skilldex: a package manager for agent skills. arXiv:2604.16911, 2026.

[28] T. Liu et al. A GovernSpec design framework for enterprise AI agents (contractual skills). arXiv:2605.22634, 2026.

[29] Y. Ge et al. Governance architecture for autonomous agent systems (LGA). arXiv:2603.07191, 2026.

[30] M. Cosler et al. NL2Spec: interactively translating unstructured natural language to temporal logics with LLMs. *CAV* 2023.

[31] Sharma et al. PROSPER: extracting protocol specifications using LLMs. *HotNets* 2023.

[32] Chen et al. TRACE: trajectory-aware comprehensive evaluation. *WWW* 2026.

[33] Surveys of LLM-as-judge reliability: positional/verbosity/self-preference biases; TNR <25%; expert agreement 64–68%. Masood 2026; Deepchecks 2025; Autorubric, arXiv:2603.00077.

[34] Anthropic. Agent Skills open standard (Dec 2025), https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills (reference implementation: https://github.com/anthropics/skills); AGENTS.md convention, https://agents.md/ (https://github.com/agentsmd/agents.md); OpenAI Model Spec, https://model-spec.openai.com/ (https://github.com/openai/model_spec); Model Context Protocol specification (2025-11-25), https://modelcontextprotocol.io/specification/2025-11-25 (https://github.com/modelcontextprotocol/modelcontextprotocol).

[35] Verified canonical URLs, license files, and exact-file permalinks for every mined real-world skill/example and cited standard referenced above, checked against the live repositories on 2026-07-12: docs/reference/MINED_SKILLS_SOURCES.md (this repository); machine-readable companion experiments/seam_bench/mining/sources.json.

[36] SafeRL-Lab (SAIL-Lab). AgenticPay: a buyer–seller negotiation benchmark for LLM agents. GitHub, 2025. MIT license. https://github.com/SafeRL-Lab/AgenticPay (commit 9740c5e).

[37] OWASP Foundation. Top 10 for agentic applications, 2026 edition.

[38] E. Zelikman, Y. Wu, J. Mu, N. D. Goodman. STaR: bootstrapping reasoning with reasoning. *NeurIPS* 2022.

[39] Z. Shao et al. DeepSeekMath: pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024. Group Relative Policy Optimization (GRPO); RL from verifiable rewards.

[40] Q. Yu et al. DAPO: an open-source LLM reinforcement learning system at scale. arXiv:2503.14476, 2025.

[41] C. Zheng et al. Group sequence policy optimization. arXiv:2507.18071, 2025.

[42] Z. Liu et al. Understanding R1-Zero-like training: a critical perspective (Dr. GRPO). arXiv:2503.20783, 2025.

[43] R. Sennrich, B. Haddow, A. Birch. Improving neural machine translation models with monolingual data (back-translation). *ACL* 2016.

[44] Verifier-checked autoformalization: Lean-ing on Quality (arXiv:2502.15795, 2025); Cycle-Consistency Fine-tuning for Lean4 (arXiv:2603.24372, 2026); miniF2F pass@k methodology.

[45] I. Y. Lee, L. D'Antoni, T. Berg-Kirkpatrick. The format tax: constrained/structured decoding can degrade reasoning and writing quality. arXiv:2604.03616, 2026.

[46] D. Banerjee, T. Suresh, S. Ugare, S. Misailovic, G. Singh. CRANE: reasoning with constrained LLM generation. *ICML* 2025; arXiv:2502.09061.

[47] P. Verga et al. Replacing judges with juries: evaluating LLM generations with a panel of diverse models (PoLL). arXiv:2404.18796, 2024.

[48] G. Kohli. Nine judges, two effective votes: correlated errors undermine LLM evaluation panels. arXiv:2605.29800, 2026.

[49] RoPoLL: robust panel of LLM judges; geometric-median aggregation with a $1/2$ breakdown point. arXiv:2606.30931, 2026.

[50] K. Liu, D. Chakraborty, A. Liggesmeyer, A. Zeller. Synthesizing precise protocol specs from natural language for effective test generation. arXiv:2511.17977, 2025.

A A validated protocol and its guard sidecar

Global type (Scribble) for the finance case, abbreviated:

```
global protocol Finance(role Ingestor, role RevenueAnalyst, role ExpenseAnalyst,
                        role TaxVerifier, role Auditor, role Writer) {
  RawRevenueData(Payload)  from Ingestor to RevenueAnalyst;
  ExpenseData(Payload)     from ExpenseAnalyst to RevenueAnalyst;
  choice at RevenueAnalyst {
    HighRevenueNotification(Analysis) from RevenueAnalyst to Auditor;
    AuditReport(Findings)             from Auditor        to TaxVerifier;
  } or {
    StandardRevenueNotification(Analysis) from RevenueAnalyst to TaxVerifier;
  }
  Approval(Decision)       from TaxVerifier to Writer;   // goal marker
  FinalReport(Document)    from Writer to Ingestor;      // goal marker, FINAL
}
```

Choice-guard sidecar (`.refn`; asserted-MPST style; stock Scribble untouched):

```
[choice at RevenueAnalyst]
when:    float(RawRevenueData) > 50000
require: HighRevenueNotification
over:    StandardRevenueNotification

[payload Approval]
assert:  Decision == 'true'  =>  next in {FinalReport}   # denied-but-debited hole
```

The guard travels through four stages: *define* (sidecar) → *state* (compiled into the contract at the decision state) → *check* (value-tracking monitor; verdict `choice_guard_violation`) → *enforce* (gate). Offline replay verification fires on high-revenue+standard-branch and on the symmetric case, stays silent on correct branches and on unevaluable guards.

B Reproducibility

All arms, cases, protocols, guards, run directories (events, prompts, per-message verdicts), the severity grader, and the metric engine are in the accompanying repository; every number in §6 names its run directory. Runner: `python experiments/scripts/case_runner.py <case> <n> --arms <keys>`. Traces are the artifact of record: re-running the deterministic evaluators on a run directory reproduces every table bit-for-bit.